AI Edge (Computing)

4. Pruning in Tensorflow

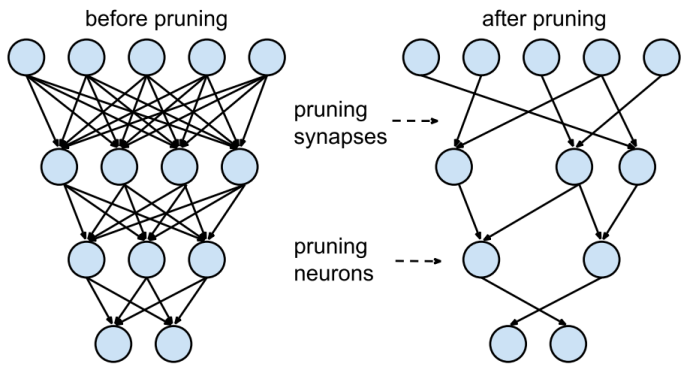
hogeschool vives

1

Pruning

General

- Two types of pruning:
  - Weight pruning (synapses)
  - Neuron pruning



hogeschool vives

2

Pruning
General

Available in Tensorflow: Magnitude-based weight pruning

Advantages?

→ Increase model sparsity
- → Easier to compress
- → Skip zeros during inference
- → Latency improvements

Disadvantages?

→ Not always better (do selective pruning)

→ Large sparsity required to achieve speed improvements (not for small models)

3

Pruning
With Tensorflow

- Make use of the "tensorflow_model_optimization" library
- Pruning is done after training
- Requires some configuration and recompilation
- Apply 'strip pruning' to remove the required variables that pruning requires during training
- !! The matrices are still the same size but are (very) sparse
  - → Compression algorithms use this as an advantage to compress the model
  - → Tensorflow Lite Micro cannot use this to its advantage (yet)
- Apply post-training quantization for additional benefits

4

Pruning

Example

```
import tensorflow as tf
import matplotlib.pyplot as plt
from tensorflow import keras
import tempfile
import numpy as np

# Load MNIST dataset
mnist = keras.datasets.mnist
(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

# Normalize the input image so that each pixel value is between 0 and 1.
train_images = train_images / 255.0
test_images = test_images / 255.0
```


5

Pruning

Example with MNIST

```
# Define the model architecture.
model = keras.Sequential([
  keras.layers.InputLayer(input_shape=(28, 28)),
  keras.layers.Reshape(target_shape=(28, 28, 1)),
  keras.layers.Conv2D(filters=12, kernel_size=(3, 3), activation='relu'),
  keras.layers.MaxPooling2D(pool_size=(2, 2)),
  keras.layers.Flatten(),
  keras.layers.Dense(10)])

# Train the digit classification model
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

model.summary()
```


6

Pruning

Example with MNIST

```
# Actually train the model
model.fit(
  train_images,
  train_labels,
  epochs=4,
  validation_split=0.1,
)

# Evaluate the model
_, baseline_model_accuracy = model.evaluate(
    test_images, test_labels, verbose=0)

print('Baseline test accuracy:', baseline_model_accuracy)
```


7

Pruning

Example with MNIST

```
# Save the model to a .h5 file
_, keras_file = tempfile.mkstemp('.h5')
tf.keras.models.save_model(model, keras_file, include_optimizer=False)
print('Saved baseline model to:', keras_file)


→ Run till here, results?
```


8

Pruning

Example with MNIST

```
# Pruning
import tensorflow_model_optimization as tfmot

prune_low_magnitude = tfmot.sparsity.keras.prune_low_magnitude

# Compute end step to finish pruning after 2 epochs.
batch_size = 128
epochs = 2
validation_split = 0.1 # 10% of training set will be used for validation set.

num_images = train_images.shape[0] * (1 - validation_split)
end_step = np.ceil(num_images / batch_size).astype(np.int32) * epochs
print("End step: %d"%(end_step))
```


9

Pruning

Example with MNIST

```
# Define model for pruning.
pruning_params = {
      'pruning_schedule': tfmot.sparsity.keras.PolynomialDecay(initial_sparsity=0.50,
                                                               final_sparsity=0.80,
                                                               begin_step=0,
                                                               end_step=end_step)
}

model_for_pruning = prune_low_magnitude(model, **pruning_params)

# `prune_low_magnitude` requires a recompile.
model_for_pruning.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])
```


10

Pruning

Example with MNIST

```
logdir = tempfile.mkdtemp()

callbacks = [
  tfmot.sparsity.keras.UpdatePruningStep(),
  # Required for logging progess (tensorboard)
  # tfmot.sparsity.keras.PruningSummaries(log_dir=logdir),
]

# Actually prune the model
model_for_pruning.fit(train_images, train_labels,
                  batch_size=batch_size, epochs=epochs, validation_split=validation_spli
                  callbacks=callbacks)
```


11

Pruning

Example with MNIST

```
# Evaluate the model
_, model_for_pruning_accuracy = model_for_pruning.evaluate(
   test_images, test_labels, verbose=0)

print('Baseline test accuracy:', baseline_model_accuracy)
print('Pruned test accuracy:', model_for_pruning_accuracy)

# export and save the pruned model to .h5
model_for_export = tfmot.sparsity.keras.strip_pruning(model_for_pruning)
model_for_export.summary()

_, pruned_keras_file = tempfile.mkstemp('.h5')
tf.keras.models.save_model(model_for_export, pruned_keras_file, include_optimizer=Fals
print('Saved pruned Keras model to:', pruned_keras_file)
```


12

Pruning

Example with MNIST

```
# Convert to .tflite and save
converter = tf.lite.TFLiteConverter.from_keras_model(model_for_export)
pruned_tflite_model = converter.convert()

_, pruned_tflite_file = tempfile.mkstemp('.tflite')

with open(pruned_tflite_file, 'wb') as f:
  f.write(pruned_tflite_model)

print('Saved pruned TFLite model to:', pruned_tflite_file)
```


13

Pruning

Example with MNIST

```
# zip the models and compare sizes
def get_gzipped_model_size(file):
  # Returns size of gzipped model, in bytes.
  import os
  import zipfile

  _, zipped_file = tempfile.mkstemp('.zip')
  with zipfile.ZipFile(zipped_file, 'w', compression=zipfile.ZIP_DEFLATED) as f:
    f.write(file)

  return os.path.getsize(zipped_file)

print("Size of baseline Keras model: %.2f bytes" % (get_gzipped_model_size(keras_file)))
print("Size of pruned Keras model: %.2f bytes" % (get_gzipped_model_size(pruned_keras_file)))
print("Size of pruned TFlite model: %.2f bytes" % (get_gzipped_model_size(pruned_tflite_file)))
```


14

Pruning

Example with MNIST

```
# create a representative dataset for extra quantisation to integers
def representative_dataset_gen():
  for input in tf.data.Dataset.from_tensor_slices(tf.cast(train_images,
tf.float32)).batch(1).take(100):
    yield [input]

converter = tf.lite.TFLiteConverter.from_keras_model(model_for_export)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_dataset_gen
quantized_and_pruned_tflite_model = converter.convert()
```


15

Pruning

Example with MNIST

```
# Save the model and compare sizes
_, quantized_and_pruned_tflite_file = tempfile.mkstemp('.tflite')

with open(quantized_and_pruned_tflite_file, 'wb') as f:
  f.write(quantized_and_pruned_tflite_model)

print('Saved quantized and pruned TFLite model to:', quantized_and_pruned_tflite_file)

print("Size of baseline Keras model: %.2f bytes" % (get_gzipped_model_size(keras_file)))
print("Size of pruned and quantized TFlite model: %.2f bytes" %
(get_gzipped_model_size(quantized_and_pruned_tflite_file)))
```


16

Pruning

Example with MNIST

```
# Compare the accuracies

import numpy as np

def evaluate_model(interpreter):
  input_index = interpreter.get_input_details()[0]["index"]
  output_index = interpreter.get_output_details()[0]["index"]

  # Run predictions on ever y image in the "test" dataset.
  prediction_digits = []
  for i, test_image in enumerate(test_images):
    if i % 1000 == 0:
      print('Evaluated on {n} results so far.'.format(n=i))
    # Pre-processing: add batch dimension and convert to float32 to
match with
    # the model's input data format.
    test_image = np.expand_dims(test_image, axis=0).astype(np.float32)
    interpreter.set_tensor(input_index, test_image)

    # Run inference.

    # … →

    # …

    interpreter.invoke()

    # Post-processing: remove batch dimension and find
the digit with highest
    # probability.
    output = interpreter.tensor(output_index)
    digit = np.argmax(output()[0])
    prediction_digits.append(digit)

  print('\n')
  # Compare prediction results with ground truth
labels to calculate accuracy.
  prediction_digits = np.array(prediction_digits)
  accuracy = (prediction_digits == test_labels).mean()
  return accuracy
```


17

Pruning

Example with MNIST

```
interpreter = tf.lite.Interpreter(model_content=quantized_and_pruned_tflite_model)
interpreter.allocate_tensors()

test_accuracy = evaluate_model(interpreter)

print('Pruned TF test accuracy:', model_for_pruning_accuracy)
print('Pruned and quantized TFLite test accuracy:', test_accuracy)
```


18

Resultaten

Model	Size (zip)	Accuracy	Timing
Baseline	78,289 kB	97,72 %	
Pruned	24,765 kB	97,57 %	245,22 ms
PTQ & Pruned	8,782 kB	97,57 %	417,51 ms
PTQ & Sparsity Pruned	11,677 kB	97,57 %	403,16 ms